COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	08/11/26	**DATE SUBMITTED:**	08/11/26
NAME:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 **Queues**

1. PROCEDURE OUTPUT

What procedures did you perform?

Raw Code:

```cpp
#include <iostream>
#include <queue>

// prototype
void display(std::queue<char> r);


int main() {
    std::queue<char> q;

    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0');

    //display queue
    std::cout<<"Displaying the queue:";
    display(q);

    //empty
    if(q.empty()){
        std::cout<<"Queue is empty!\n";
    } else {
        std::cout<<"Queue has elements.\n";
    }

    //size
    std::cout<<"The amount of elements in the queue is "<<q.size()<<".\n";
```

```
32      //front
33      std::cout<<"Element at the front: "<<q.front()<<"\n";
34
35      //back
36      std::cout<<"Element at the back: "<<q.back()<<"\n";
37
38      //pop
39      std::cout<<"Popping...\n";
40      q.pop();
41
42      //display queue
43      std::cout<<"Displaying the queue:";
44      display(q);
45
46      //push
47      std::cout<<"Pushing...\n";
48      q.push('Z');
49
50      //display queue
51      std::cout<<"Displaying the queue:";
52      display(q);
53
```

extensions are disabled due to version incompatibility.

```
55      // ### CHALLENGE
56      std::cout<<std::endl; // spacing
57
58      // Removing the queue!
59      std::cout<<"Destroying the queue!\n";
60      while(!q.empty()){
61          q.pop();
62      }
63
64      // should be empty two checks; displaying the queue and if empty
65      //display queue
66      std::cout<<"Displaying the queue:";
67      display(q);
68
69      //empty
70      if(q.empty()){
71          std::cout<<"Queue is empty!\n";
72      } else {
73          std::cout<<"Queue has elements.\n";
74      }
75
76      return 0;
77  }
```

```
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA5p1\ILOA> cd
Displaying the queue: c p e 0 1 0
Queue has elements.
The amount of elements in the queue is 6.
Element at the front: c
Element at the back: 0
Popping...
Displaying the queue: p e 0 1 0
Pushing...
Displaying the queue: p e 0 1 0 Z

Destroying the queue!
Displaying the queue:
Queue is empty!
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA5p1\ILOA>
```

Code Analysis:

The code above shows the use of queue functions in the standard template library. The final output is as follows; displaying the queue, checking if it's empty, seeing the element at the front then back, popping an element (removing the front), displaying it, pushing an element (adding to the back), followed by the challenge which is destroying the queue. The destruction of the queue is done with two checks; the **q.display()** and **q.empty()** with an if condition to check if the queue is truly empty. Additionally, the destruction of the queue was done with a while loop combined with the pop function.

Output of ILO A

Raw Code for mainILOB1:

```
#include <iostream>
#include "queuell.h"

int main(){
    qNode<char> *front = nullptr;
    qNode<char> *back = nullptr;

    // Enqueue
    std::cout<<"Enqueuing: \n";
    enqueue(&front, &back, 'M');
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue(&front, &back, 'J');
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue(&front, &back, 'O');
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue(&front, &back, 'M');
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue(&front, &back, 'Z');
    std::cout<<front->data<<" "<<back->data<<std::endl;

    std::cout<<std::endl; // Spacing

```

```
23      // Dequeue
24      std::cout<<"Dequeuing: \n";
25      dequeue(&front, &back);
26      std::cout<<front->data<<" "<<back->data<<std::endl;
27
28      std::cout<<std::endl; // Spacing
29
30      // Display
31      std::cout<<"Printing all elements: \n";
32      display(front);
33
34      std::cout<<std::endl; // Spacing
35
36      // Front
37      std::cout<<"Front Element: ";
38      std::cout<<frontFind(front);
39
40      return 0;
41  }
```

```
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA5p1\ILOB> c
Enqueuing:
M M
M J
M O
M M
M Z

Dequeuing:
J Z

Printing all elements:
J -> O -> M -> Z -> nullptr

Front Element: J
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA5p1\ILOB>
```

Code Analysis:

The code above in main declares and initializes the front and back nodes pointing to null while including the calling of the queue functions that we created in our custom library. The output in the terminal shows the front and back elements each time a new element is inputted into the queue, which is partly the same on how dequeuing is presented, followed by the printing of all elements and front.

Raw Code (queuell.h):

Headers, initial node class, and node creation template.

```
1   #include <iostream>
2   #ifndef QUEUELL_H
3   #define QUEUELL_H
4
5   template <typename T>
6   class qNode{
7       public:
8       T data;
9       qNode *next;
10  };
11
12  // creating a new node
13  template <typename T>
14  qNode<T> *new_node(T newData){
15      qNode<T>* newNodePtr = new qNode<T>;
16      newNodePtr->data = newData;
17      newNodePtr->next = nullptr;
18      return newNodePtr;
19  }
20
```

Code Analysis:

The code above shows the header and the library file being defined, the class created for the node bearing a pointer and a dynamically assigned type of variable. Additionally, the creation of a new node is also included with accepting the data and setting it up as well as initializing its **next** to be defaulted in pointing towards **nullptr**.

enqueue()

```
//ENQUEUE
template<typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
    // create a new node
    qNode<T> *newNodePtr = new_node(newData);

    //inserting to an empty queue
    if((*frontPtr)==nullptr && (*backPtr)==nullptr){
        (*frontPtr) = newNodePtr;
        (*backPtr) = newNodePtr;
    }
    //inserting an item into a non empty queue
    else {
        (*backPtr)-> next = newNodePtr;
        (*backPtr) = newNodePtr;
    }
}
```

Code Analysis:

This function allows a node to be added to the queue while having checks for cases such that if the queue that is used is empty or not using an if-else statement.

dequeue()

```
//DEQUEUE
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    // create a temporary node to store the node to be deleted
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);

    // check if the queue is only one node
    if((*frontPtr)->next==nullptr && (*backPtr)->next==nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }

    // deleting of a node
    (*frontPtr)=deleteNode->next;
    delete deleteNode;
}
```

Code Analysis:

<table>
<tr><td></td><td>Similar to enqueue, dequeue also has a check for when there is only one node in the element. It makes use of storing a temporary node in order to handle its deletion process.</td></tr>
<tr><td rowspan="3">display()</td><td>

```
// DISPLAY - display all elements in the queue
template <typename T>
void display(qNode<T>* frontPtr) {
    qNode<T>* currentHead = frontPtr;
    while (currentHead != nullptr) {
        std::cout << currentHead->data << " -> ";
        currentHead = currentHead->next;
    }
    std::cout << "nullptr" << std::endl;
}
```

</td></tr>
<tr><td>Code Analysis:</td></tr>
<tr><td>The code above shows the function, display() which makes use of also using a temporary variable named currentHead that uses frontPtr as its first reference. Using a while-loop, it iterates through the queue by printing and setting the following pointer to the next, eventually stopping when the next pointer is equal to nullptr which would mean that it had finally reached the tail.</td></tr>
<tr><td rowspan="3">frontFind()</td><td>

```
72  // FRONT - return the front element of the queue
73  template <typename T>
74  T frontFind(qNode<T>* frontPtr) {
75      return frontPtr->data;
76  }
77
78  #endif
```

</td></tr>
<tr><td>Code Analysis:</td></tr>
<tr><td>The function above simply takes the front pointer, accesses its data, and returns it for printing. There is nothing much else to explain.</td></tr>
</table>

Output of ILO B.1

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

mainILOB2:

mainILOC.cpp 5 | queueArr.h ILOB | mainILOB2.cpp | classes.h 1 | queueArr.h ILOC

ILOB > mainILOB2.cpp > main()

```cpp
#include <iostream>
#include "queueArr.h"

int main() {

    // INITIALIZATION
    queueArr<int> q(7);

    // CHECK IF EMPTY
    q.Empty();

    // ENQUEUE
    std::cout << "\nEnqueuing 1, 2, 3, 4, 5, 6, and 7..." << std::endl;
    q.Enqueue(1);
    q.Enqueue(2);
    q.Enqueue(3);
    q.Enqueue(4);
    q.Enqueue(5);
    q.Enqueue(6);
    q.Enqueue(7);

    std::cout << "Size of queue: " << q.Size() << std::endl;
    std::cout << "Front of the queue: " << q.Front() << std::endl;
    std::cout << "Back of the queue: " << q.Back() << std::endl;

    // DEQUEUE AND DISPLAY
    std::cout << "\n Displaying, dequeueing one element, then displaying again..." << std::endl;
    q.Display();
    q.Dequeue();
    q.Display();

    std::cout << "Displaying, enqueuing one element (67), then displaying again..." << std::endl;
    q.Display();
    q.Enqueue(67);
    q.Display();
    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;

```

```
    // COPY CONSTRUCTOR
    std::cout << "\nTesting Copy Constructor..." << std::endl;
    queueArr<int> qAnotherOne = q;
    std::cout << "The Copied Queue: ";
    qAnotherOne.Display();

    // COPY ASSIGNMENT OPERATOR
    std::cout << "\nTesting Copy Assignment Operator..." << std::endl;
    queueArr<int> qAssignment(7);
    qAssignment = q;
    std::cout << "Assigned Queue: ";
    qAssignment.Display();

    // CLEARING
    std::cout << "\nClearing..." << std::endl;
    q.Clear();
    std::cout << "Is queue empty? " << (q.Empty() ? "Yes" : "No") << std::endl; // Ternary operator
    q.Display();

    return 0;
}
```

```
PS C:\Users\Jomz\Documents\CPE010\MEDINA CPE010 - CPE21S1\HOA5p1\ILOB> cd "c
 mainILOB2 } ; if ($?) { .\mainILOB2 }

Enqueuing 1, 2, 3, 4, 5, 6, and 7...
Size of queue: 7
Front of the queue: 1
Back of the queue: 7

 Displaying, dequeueing one element, then displaying again...
Front -> 1 2 3 4 5 6 7 <- Back
Front -> 2 3 4 5 6 7 <- Back
Displaying, enqueuing one element (67), then displaying again...
Front -> 2 3 4 5 6 7 <- Back
Front -> 2 3 4 5 6 7 67 <- Back
Front: 2
Back: 67

Testing Copy Constructor...
The Copied Queue: Front -> 2 3 4 5 6 7 67 <- Back

Testing Copy Assignment Operator...
Assigned Queue: Front -> 2 3 4 5 6 7 67 <- Back

Clearing...
Is queue empty? Yes
The queue is empty!
PS C:\Users\Jomz\Documents\CPE010\MEDINA CPE010 - CPE21S1\HOA5p1\ILOB>
```

COMPUTER ENGINEERING

QUEZON CITY

<table>
<tr><th colspan="2">Raw Code (queueArr.h):</th></tr>
<tr><td rowspan="3">Headers, initial array class, private attributes</td><td>

```
#include <iostream>
#ifndef QUEUEARR_H
#define QUEUEARR_H

template <typename T>
class queueArr{
private:
    T* q_array;
    int q_capacity;
    int q_size;
    int q_front;
    int q_back;
```

</td></tr>
<tr><td>Code Analysis:</td></tr>
<tr><td>The code above shows the header and the library file being defined, the class created for the array-based queue, and the private attributes.</td></tr>
<tr><td rowspan="3">Constructor</td><td>

```
public:
    // CONSTRUCTOR
    queueArr(int max = 7) { // Capacity to match the 7 elements in the lab manual as default parameter
        q_capacity = (max > 0) ? max : 7; // Ternary operator (makes sure capacity is not set to 0)
        q_array = new T[q_capacity];
        Clear();
    }
```

</td></tr>
<tr><td>Code Analysis:</td></tr>
<tr><td>The constructor initializes the empty queue object which sets the max amount or capacity to a default 7 if not specified. It's dynamically allocated on the heap.</td></tr>
<tr><td rowspan="3">Destructor</td><td>

```
    // DESTRUCTOR
    ~queueArr() {
        delete[] q_array;
    }
```

</td></tr>
<tr><td>Code Analysis:</td></tr>
<tr><td>The destructor handles the cleanup of the memory when the queue goes out of the scope, calling delete on our qArray to deallocate the dynamically allocated memory in order to prevent memory leaks.</td></tr>
</table>

Copy Constructor

```
27      // COPY CONSTRUCTOR
28      queueArr(const queueArr<T>& other) {
29          q_capacity = other.q_capacity;
30          q_size = other.q_size;
31          q_front = other.q_front;
32          q_back = other.q_back;
33          q_array = new T[q_capacity];
34
35          for (int i = 0; i < q_capacity; ++i) {
36              q_array[i] = other.q_array[i];
37          }
38      }
```

Code Analysis:

The code above shows the copy constructor which creates a copy of our existing queue object, copying all of the member variables.

Copy Assignment Operator

```
40  // COPY ASSIGNMENT
41      queueArr<T>& operator=(const queueArr<T>& other) {
42          if (this != &other) {
43              delete[] q_array;
44
45              q_capacity = other.q_capacity;
46              q_size = other.q_size;
47              q_front = other.q_front;
48              q_back = other.q_back;
49              q_array = new T[q_capacity];
50
51              for (int i = 0; i < q_capacity; ++i) {
52                  q_array[i] = other.q_array[i];
53              }
54          }
55          return *this;
56      }
```

Code Analysis:

The code above shows the copy assignment operator which replaces the object's current data with another copy of another queue.

<table>
<tr><td>Declarations</td><td>

```
58      // DECLARATIONS
59      bool Empty() const;
60      int Size() const;
61      void Clear();
62      T Front();
63      const T& Front() const;
64      T Back();
65      const T& Back() const;
66      void Enqueue(const T& val);
67      void Dequeue();
68      void Display() const;
69  };
```

</td></tr>
<tr><td rowspan="3">Empty()</td><td>

```
71  // EMPTY
72  template <typename T>
73  bool queueArr<T>::Empty() const {
74      return q_size == 0;
75  }
```

</td></tr>
<tr><td>Code Analysis:</td></tr>
<tr><td>This function checks whether the queue holds any elements or not by using a comparison operator.</td></tr>
<tr><td rowspan="3">Size()</td><td>

```
77  // SIZE
78  template <typename T>
79  int queueArr<T>::Size() const {
80      return q_size;
81  }
```

</td></tr>
<tr><td>Code Analysis:</td></tr>
<tr><td>Simply returns the value of q_size when called.</td></tr>
</table>

Clear()

```
83  // CLEAR
84  template <typename T>
85  void queueArr<T>::Clear() {
86      q_size = 0;
87      q_front = 0;
88      q_back = q_capacity - 1;
89  }
90
```

Code Analysis:

This function "clears" the queue by setting the size and front to 0 as well as decrementing the capacity and setting it as the back.

Front()

```
91  // FRONT
92  template <typename T>
93  T queueArr<T>::Front() {
94      if (Empty()) {
95          std::cout<<"The queue is empty!";
96          return T{};
97      }
98      return q_array[q_front];
99  }
100
```

Code Analysis:

This function simply returns the front element by using **q_front**.

Back()

```
101 // BACK
102 template <typename T>
103 T queueArr<T>::Back() {
104     if (Empty()) {
105         std::cout<<"The queue is empty!";
106         return T{};
107     }
108     return q_array[q_back];
109 }
110
```

Code Analysis:

This function does the same thing as **front()** but accesses the back instead.

Enqueue()

```
111  // ENQUEUE
112  template <typename T>
113  void queueArr<T>::Enqueue(const T& val) {
114      if (q_size == q_capacity) {
115          std::cout<<"The queue is full!";
116          return;
117      }
118      q_back = (q_back + 1) % q_capacity;
119      q_array[q_back] = val;
120      q_size++;
121  }
```

Code Analysis:

This function inserts a new value into the back of the queue, incrementing the size accordingly.

Dequeue()

```
123  // DEQUEUE
124  template <typename T>
125  void queueArr<T>::Dequeue() {
126      if (Empty()) {
127          std::cout<<"The queue is empty!";
128          return;
129      }
130      q_front = (q_front + 1) % q_capacity;
131      q_size--;
132  }
```

Code Analysis:

This function removes the first element and decrements the size of the queue accordingly.

Display()

```
134  // DISPLAY
135  template <typename T>
136  void queueArr<T>::Display() const {
137      if (Empty()) {
138          std::cout<<"The queue is empty!";
139          return;
140      }
141      for (int i = 0; i < q_size; ++i) {
142          int indexNum = (q_front + i) % q_capacity;
143          std::cout << q_array[indexNum] << " ";
144      }
145  }
146
147  #endif
```

Code Analysis:

Finally, our display function does a check of whether or not the queue is empty, before proceeding to display the queue array object with a simple **for loop**.

Output of ILO B.2

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Chosen Design: ARRAY-BASED QUEUE
classes.h (contains my Job and Printer classes)

ILOC > C classes.h > Job > Job()

```
1   #ifndef CLASSES_H
9   class Job {
13      int numOfPages;
14
15  public:
16      // DEFAULT CONSTRUCTOR & CONSTRUCTOR
17      Job()
18      {
19          jobID = 0;
20          username = "";
21          numOfPages = 0;
22      }
23
24      Job(int id, std::string name, int pages) {
25          jobID = id;
26          username = name;
27          numOfPages = pages;
28      }
29
30      // GETTERS
31      int getJobID() {
32          return jobID;
33      }
34
35      std::string getName() {
36          return username;
37      }
38
39      int getNumOfPages() {
40          return numOfPages;
41      }
```

```
    // DISPLAY
    void displayJob() {
        std::cout << "[ #" << jobID << " - User: " << username << " - Pages: " << numOfPages << " ]";
    }
};


// PRINTER CLASS
class Printer {
private:
    queueArr<Job> jobsList;

public:
    // CONSTRUCTOR (using member initializer)
    Printer(int max = 10): jobsList(max){}

    // REQUEST A JOB
    void addJob(int jobID, std::string username, int numOfPages){
        Job newJob(jobID, username, numOfPages);

        jobsList.Enqueue(newJob);

        std::cout << "New Request: ";
        newJob.displayJob();
        std::cout << " added to queue." << std::endl;
    }
```

```
    // PROCESS JOB (oldest)
    void processNext()
    {
        if (jobsList.Empty())
        {
            std::cout << "Idle... No pending jobs to print.\n";
            return;
        }

        Job currentJob = jobsList.Front();

        std::cout << "Printing: ";
        currentJob.displayJob();
        std::cout << " -> Printing complete!\n";

        jobsList.Dequeue();
    }
```

```
88      // DO ALL
89      void processAll()
90      {
91          std::cout << "\n-- Remaining Jobs --\n";
92
93          while (!jobsList.Empty()) {
94              processNext();
95          }
96
97          std::cout << "-- All jobs completed. --\n\n";
98      }
99
100     // Returns the number of jobs waiting
101     int getPendingCount()
102     {
103         return jobsList.Size();
104     }
105 };
106
107 #endif
```

mainILOC.cpp

mainILOC.cpp 5 | queueArr.h ILOB | queueArr.h ILOC | mainILOB2.exe | mainILOB2.cpp | classes.h 1

ILOC > mainILOC.cpp > ...

```cpp
#include <iostream>
#include "classes.h"

int main() {
    Printer myPrinter(5); // Create printer object with max space of 5 job orders

    std::cout << "ILO C - PRINTER TEST" << std::endl;

    // Multiple requests from multiple users
    std::cout << "-- Submitting Print Requests --" << std::endl;
    myPrinter.addJob(001, "Jomz", 67);
    myPrinter.addJob(002, "Ryle", 6);
    myPrinter.addJob(003, "Medina", 7);
    myPrinter.addJob(004, "CPE21S1", 76);

    std::cout << "\nTotal Pending Jobs: " << myPrinter.getPendingCount() << "\n" << std::endl;

    // Doing the first two jobs
    std::cout << "-- Processing First Two Requests --" << std::endl;
    myPrinter.processNext();
    myPrinter.processNext();

    // Enqueue a print request
    std::cout << "\n-- New Print Request Arrives --" << std::endl;
    myPrinter.addJob(005, "CPE010", 22);

    // Do all the jobs
    myPrinter.processAll();

    // Try printing while the queue is empty (must show that its in idle and that there are no pending jobs to print)
    std::cout << "-- Attempting to print from empty queue --" << std::endl;
    myPrinter.processNext();

    return 0;
}
```

```
ILO C - PRINTER TEST
-- Submitting Print Requests --
New Request: [ #1 - User: Jomz - Pages: 67 ] added to queue.
New Request: [ #2 - User: Ryle - Pages: 6 ] added to queue.
New Request: [ #3 - User: Medina - Pages: 7 ] added to queue.
New Request: [ #4 - User: CPE21S1 - Pages: 76 ] added to queue.

Total Pending Jobs: 4

-- Processing First Two Requests --
Printing: [ #1 - User: Jomz - Pages: 67 ] -> Printing complete!
Printing: [ #2 - User: Ryle - Pages: 6 ] -> Printing complete!

-- New Print Request Arrives --
New Request: [ #5 - User: CPE010 - Pages: 22 ] added to queue.

-- Remaining Jobs --
Printing: [ #3 - User: Medina - Pages: 7 ] -> Printing complete!
Printing: [ #4 - User: CPE21S1 - Pages: 76 ] -> Printing complete!
Printing: [ #5 - User: CPE010 - Pages: 22 ] -> Printing complete!
-- All jobs completed. --

-- Attempting to print from empty queue --
Idle... No pending jobs to print.
PS C:\Users\Jomz\Documents\CPE010\MEDINA CPE010 - CPE21S1\HOA5p1\ILOC>
```

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).

```
class Job {
private:
    int jobID;
    std::string username;
    int numOfPages;

public:
    // DEFAULT CONSTRUCTOR & CONSTRUCTOR
    Job()
    {
        jobID = 0;
        username = "";
        numOfPages = 0;
    }

    Job(int id, std::string name, int pages) {
        jobID = id;
        username = name;
        numOfPages = pages;
    }

    // GETTERS
    int getJobID() {
        return jobID;
    }

    std::string getName() {
        return username;
    }

    int getNumOfPages() {
        return numOfPages;
    }

    // DISPLAY
    void displayJob() {
        std::cout << "[ #" << jobID << " - User: " << username << " - Pages: " << numOfPages << " ]";
    }
};
```

2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.

```cpp
// PRINTER CLASS
class Printer {
private:
    queueArr<Job> jobsList;

public:
    // CONSTRUCTOR (using member initializer)
    Printer(int max = 10): jobsList(max){}

    // REQUEST A JOB
    void addJob(int jobID, std::string username, int numOfPages){
        Job newJob(jobID, username, numOfPages);

        jobsList.Enqueue(newJob);

        std::cout << "New Request: ";
        newJob.displayJob();
        std::cout << " added to queue." << std::endl;
    }

    // PROCESS JOB (oldest)
    void processNext()
    {
        if (jobsList.Empty())
        {
            std::cout << "Idle... No pending jobs to print.\n";
            return;
        }

        Job currentJob = jobsList.Front();

        std::cout << "Printing: ";
        currentJob.displayJob();
        std::cout << " -> Printing complete!\n";

        jobsList.Dequeue();
    }

    // DO ALL
    void processAll()
    {
        std::cout << "\n-- Remaining Jobs --\n";

        while (!jobsList.Empty()) {
            processNext();
        }

        std::cout << "-- All jobs completed. --\n\n";
    }

    // Returns the number of jobs waiting
    int getPendingCount()
    {
        return jobsList.Size();
    }
};
```

3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.

```
// REQUEST A JOB
void addJob(int jobID, std::string username, int numOfPages){
    Job newJob(jobID, username, numOfPages);

    jobsList.Enqueue(newJob);

    std::cout << "New Request: ";
    newJob.displayJob();
    std::cout << " added to queue." << std::endl;
}
```

4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.

```
ILO C - PRINTER TEST
-- Submitting Print Requests --
New Request: [ #1 - User: Jomz - Pages: 67 ] added to queue.
New Request: [ #2 - User: Ryle - Pages: 6 ] added to queue.
New Request: [ #3 - User: Medina - Pages: 7 ] added to queue.
New Request: [ #4 - User: CPE21S1 - Pages: 76 ] added to queue.

Total Pending Jobs: 4

-- Processing First Two Requests --
Printing: [ #1 - User: Jomz - Pages: 67 ] -> Printing complete!
Printing: [ #2 - User: Ryle - Pages: 6 ] -> Printing complete!

-- New Print Request Arrives --
New Request: [ #5 - User: CPE010 - Pages: 22 ] added to queue.

-- Remaining Jobs --
Printing: [ #3 - User: Medina - Pages: 7 ] -> Printing complete!
Printing: [ #4 - User: CPE21S1 - Pages: 76 ] -> Printing complete!
Printing: [ #5 - User: CPE010 - Pages: 22 ] -> Printing complete!
-- All jobs completed. --

-- Attempting to print from empty queue --
Idle... No pending jobs to print.
PS C:\Users\Jomz\Documents\CPE010\MEDINA CPE010 - CPE21S1\HOA5p1\ILOC>
```

5. Design Defense:

In the supplementary code, I decided to create a separate library class that I'd just import in the main.cpp file which contains all the classes I'd need so as to not clutter the main code up. Instead of picking a linked-list based queue, I decided on using the array-based one created in ILO B. This is because the printer has a maximum capacity that is fixed for jobs that are pending. Using arrays allows me to store the jobs in a very simple and more organized way where the circular queue makes it possible to reuse positions after the jobs are removed. When comparing this to the linked list, using arrays is much simpler to implement for the activity because our printer already has the maximum jobs that it could possibly hold.

Output of ILO C.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.